

Chaos Control Via Neural Networks

Using Radial Basis Functions

Serdar Iplikci ¹, Yagmur Denizhan ²

¹ Research Assistant, e-mail: iplikcis@boun.edu.tr, phone: +90-212-263 15 40/1855

² Associate Professor, e-mail: denizhan@boun.edu.tr, phone: +90-212-263 15 40/1850
Electrical and Electronics Eng. Dept., Bogazici University, 80815 Bebek Istanbul / TURKEY

Abstract. In this work, a targeting method to be used together with the local OGY control is suggested. In order to reduce the typical drawback of the OGY control, i.e. the long duration usually required for a chaotic system to reach the close neighbourhood of the chosen target –an unstable equilibrium point or an unstable periodic orbit-, additional activation regions are introduced, starting from which the system can be steered towards the target within a few steps applying small perturbations to the control parameter. As in conventional OGY control, the a priori knowledge of the system dynamics is not required. The suggested Extended Control Regions (ECR) method has been implemented with a Neural Network using Radial Basis Functions on several chaotic systems and the successful reduction in the average reaching time has been demonstrated.

1. INTRODUCTION

In the recent decades, the control of chaotic systems has gained much attention. In particular, Ott, Grebogi and Yorke [1] have introduced a control scheme, according to which the chosen target (either an unstable equilibrium point or an unstable periodic orbit of the system) can be stabilised applying small variations to the control parameter. Several extensions and modifications of this local approximation method using more than one control parameters and applied to higher dimensional systems have been proposed in the subsequent years [2-9].

The fact that the OGY method and its extensions do not require any a priori knowledge about the system dynamics and allow parameter-based-control with small control power expenditure, constitutes the most important advantages of these methods.

The major drawback of these methods is the usually long reaching time until the system visits a close neighbourhood of the target and the control can be applied. The problem of reducing this reaching time is referred to as the targeting problem and several targeting methods have been suggested in the literature [10-12]. Furthermore, neural networks have also been utilised for various purposes in controlling chaotic dynamic systems [13-20].

In this work a novel targeting approach named Extended Control Regions (ECR) has been proposed, which makes use of the multi-control-parameter version of the OGY method. The ECR targeting method has been realised using a single Neural Network based on Radial Basis Functions. It has been demonstrated that the ECR method successfully reduces the reaching time while maintaining the major advantages of the OGY method.

2. THE OGY METHOD

In all versions and modifications of the OGY method [1-9] it is assumed that:

1. There is no a priori knowledge available about the system dynamics. Instead, the OGY control is accomplished exploiting the information extracted from experimental data.
2. One or several of the system parameters can be varied about their nominal values and thus used for control purposes.

First, the available experimental data is used in order to detect the unstable equilibrium points and unstable periodic orbits embedded into the strange attractor of the chaotic system under consideration. It should be noted that there are usually many, sometimes even infinitely many such equilibrium points and periodic orbits.

In order to detect a periodic orbit of a continuous-time system usually the Poincaré section method is employed. According to this method, first a Poincaré surface is chosen and a direction is assigned to it. The locations at which the system trajectories pierce the surface at the assigned direction are recorded. Thus a map of the form

$$\mathbf{z}_{n+1} = \mathbf{f}(\mathbf{z}_n) \quad (1)$$

where $\mathbf{z}_n$ denotes the piercing location on the Poincaré surface at the n^{th} piercing. It should be noted that the Poincaré map of a N -dimensional continuous-time system is $N-1$ dimensional. The equilibrium points of the Poincaré map correspond to the periodic orbits of the original continuous-time system. Hence, the detection and stabilisation of an unstable equilibrium point of a discrete-time system and that of an unstable periodic orbit of a continuous-time system using the Poincaré method are equivalent. Here the OGY method will be presented for these cases. A continuous version of the OGY method for the stabilisation of the unstable equilibrium points of continuous-time systems also exists in the literature [21-22].

Another approach, called the delay coordinates method, has to be employed if only one of the states of an N -dimensional continuous-time chaotic system is measurable. If we denote this measurable state by $x(t)$, choosing an appropriate delay time T and delay dimension M , a delay coordinate vector $\mathbf{x}(t)$ can be reconstructed as given in (2).

$$\mathbf{x}(t) = [x(t), x(t-T), x(t-2T), \dots, x(t-MT)] \quad (2)$$

If T and M are appropriately chosen [23-24], there exists a diffeomorphism between the dynamics of the reconstructed new state vector and the real state vector [25]. Hence the equilibrium points and periodic orbits in the real system have their counterparts in the delay coordinate system with the same stability properties. The delay coordinate method allows the application of the OGY control to multi-dimensional chaotic systems where one state is measurable.

After the unstable equilibrium points or unstable periodic orbits of the system are observed from the data, one of them, which exhibits a desirable behaviour, is chosen as the target. The fact that the OGY method tries to stabilise the system at one of its own unstable equilibrium modes, constitutes a basic deviation from the classical control approach, where the system is forced to follow an externally given reference trajectory. The huge number of unstable equilibrium modes embedded into the strange attractor of a chaotic system usually provides a sufficiently rich repertoire for a desirable target.

Once the target has been determined, data gathered under small control parameter variations are used in order to obtain a linear model (3) of the system dynamics (i.e. the system dynamics of a discrete-time system or the Poincaré map of a continuous-time system) in the close neighbourhood of the target.

$$\mathbf{z}_{n+1} - \mathbf{z}^* = \underline{\underline{\mathbf{A}}} (\mathbf{z}_n - \mathbf{z}^*) + \underline{\underline{\mathbf{B}}} (\mathbf{p} - \mathbf{p}_{\text{nom}}) \quad (3)$$

Here, $\mathbf{z}_n$ denotes the state vector of the system at the n^{th} step, $\mathbf{z}^*$ denotes the target state and $\mathbf{p}_{\text{nom}}$ the nominal parameter vector. $\underline{\underline{\mathbf{A}}}$ and $\underline{\underline{\mathbf{B}}}$ are the matrices that determine the local linear model.

The OGY controller is activated only when the system trajectory enters a close neighbourhood of the target ($\mathbf{z}^*$) and supplies the necessary parameter values to keep the system very close to the target.

The control of a system using parameter variations is formally different than the classical control (4) approach using an external input.

$$\mathbf{z}_{n+1} = \mathbf{f}(\mathbf{z}_n, \mathbf{p}_{\text{nom}}) + \underline{\underline{\mathbf{B}}} \mathbf{u} \quad (4)$$

where $\mathbf{u}$ is the external control input and $\underline{\underline{\mathbf{B}}}$ can be designed freely in order to achieve the desired control performance.

However, comparing (3) and (4) one can easily observe that $(\mathbf{p} - \mathbf{p}_{\text{nom}})$ can be considered as the external input. Yet in the OGY case $\underline{\underline{\mathbf{B}}}$ is determined by the system's own local dynamics.

There exist such application areas that allow small parameter variations rather than an external control input. The OGY method is applicable to such systems. The fact that the OGY method does not require any a priori knowledge about the system dynamics and can achieve the control task using small parameter variations (small control action) constitutes its main advantages.

On the other hand, the major drawback of the OGY method is due to the long waiting time until the system trajectory enters a close neighbourhood of the target.

3. EXTENDED CONTROL REGIONS (ECR) APPROACH

The reaching time until the system enters the close neighbourhood of the target (the OGY region) may be quite long depending on the initial conditions and the size of the strange attractor. Therefore, most chaotic systems require

targeting, i.e. a way of steering the system from any initial point towards the OGY region. Since targeting requires global information about the strange attractor, local approximations are not useful any more. The proposed Extended Control Regions (ECR) method does not use a complete model of the strange attractor, yet requires some information scattered about the whole attractor. Therefore it is necessary to make use of a tool that has good approximation capabilities over a larger region.

As in OGY control, it is assumed that a priori knowledge about system dynamics is not available, and that only the system parameter(s) can be used for control purposes. Moreover, the ECR method is also applicable to delay coordinates as described in Section 2.

The ECR method will be presented for N-dimensional discrete-time chaotic systems with r control parameters only. As explained in Section 2, this model covers both continuous-time systems with an unstable periodic orbit as the target (via Poincaré section method) and discrete-time systems. Such a system is represented by (5).

$$\mathbf{z}_{n+1} = \mathbf{G}(\mathbf{z}_n, \mathbf{p}), \quad (5)$$

where $\mathbf{z}$ is the $N \times 1$ state vector, and $\mathbf{p}$ is the $r \times 1$ control parameter vector. Assume that this system has an equilibrium point at $\mathbf{z}^*$ described by,

$$\mathbf{z}^* = \mathbf{G}(\mathbf{z}^*, \mathbf{p}_{\text{nom}}), \quad (6)$$

where $\mathbf{p}_{\text{nom}}$ is the $r \times 1$ vector of nominal parameter values. Each control variable (parameter) can be changed within its allowable range, i.e.

$$\Pi = \{p^i : p_{\text{nom}}^i - \delta p_{\text{max}}^i < p^i < p_{\text{nom}}^i + \delta p_{\text{max}}^i, i=1, \dots, r\} \quad (7)$$

where, δp_{max}^i is the maximum allowable parameter change for the i^{th} parameter.

The basic idea behind the ECR method is to identify different regions (S_i , $i=0, 1, \dots, K$) of the phase space, where the controller will be activated either for targeting or local (OGY) control purposes. Hence, the activation region of the controller is extended from S_0 (as used in the OGY control) to the union of all S_i 's ($i=0, 1, \dots, K$). These regions can be defined as follows:

Definition 1:

If a system state $\mathbf{z}_n$ obeys (8) then $\mathbf{z}_n \in S_0$.

$$|\mathbf{z}_n - \mathbf{z}^*| < \delta, \text{ and } |\mathbf{G}(\mathbf{z}_n, \mathbf{p}) - \mathbf{z}^*| < \delta \text{ with } \mathbf{p} \in \Pi \quad (8)$$

In other words, starting from any $\mathbf{z}_n \in S_0$ the system can be kept within the δ neighbourhood of $\mathbf{z}^*$ at the next step using a $\mathbf{p} \in \Pi$.

Definition 2:

$\mathbf{z}_n \in S_1$ if it satisfies (9).

$$\mathbf{z}_n \notin S_0, \text{ and } \mathbf{G}(\mathbf{z}_n, \mathbf{p}_n) \in S_0, \quad \mathbf{p}_n \in \Pi \quad (9)$$

Similarly,

$\mathbf{z}_n \in S_2$ if it satisfies (10).

$$\mathbf{z}_n \notin S_0, \text{ and } \mathbf{G}(\mathbf{G}(\mathbf{z}_n, \mathbf{p}_n), \mathbf{p}_{n+1}) \in S_0, \quad \mathbf{p}_n, \mathbf{p}_{n+1} \in \Pi \quad (10)$$

The definition can be generalised as follows:

$\mathbf{z}_n \in S_i$ if it satisfies (11).

$$\mathbf{z}_n \notin S_0, \text{ and } \mathbf{G}^{(i)}(\mathbf{z}_n, \mathbf{p}_n, \mathbf{p}_{n+1}, \dots, \mathbf{p}_{n+i}) \in S_0, \quad \mathbf{p}_n, \dots, \mathbf{p}_{n+i} \in \Pi \quad (11)$$

where

$$\mathbf{G}^{(i)}(\mathbf{z}_n, \mathbf{p}_n, \mathbf{p}_{n+1}, \dots, \mathbf{p}_{n+i}) = \mathbf{G}(\mathbf{G}(\dots(\mathbf{G}(\mathbf{z}_n, \mathbf{p}_n)\mathbf{p}_{n+1}), \dots, \mathbf{p}_{n+i})) \quad (12)$$

In other words, $\mathbf{z}_n \in S_i$ if it is not in S_0 but can be steered to S_0 in i steps applying control parameters within the allowable range. If the dynamics of the system in S_i 's is sufficiently modelled, the reaching time can be reduced at an increasing rate as K increases.

Therefore, ideally the system can be directed from any region S_i into S_0 within i steps by applying parameter values within the allowable range; i.e. starting from S_i the system first goes to S_{i-1} , then to S_{i-2} , S_{i-3} etc. It should be noted that theoretically a region S_i does not need to be simply connected or connected; furthermore, successive regions can even be interlaced. As a matter of fact the real regions have most of the time a fractal geometry.

On the other hand, any practical model of S_i 's (e.g. the internal model obtained by Neural Networks when trained by the training data) will usually fail to capture their fractal nature. Consequently, the practical models of S_i 's will only be approximations of the real S_i 's. However, as can be observed from the simulation results, this does not seriously affect the performance of the controller.

The sizes of S_i 's depend both on the width of allowable parameter values and the number of control parameters. The more control parameters are used and the larger the allowable intervals for parameters are, the larger are the

activation regions. However, the number of activation regions and the magnitudes of allowable parameter changes cannot be chosen arbitrarily large due to several reasons.

(a) The choice of too many activation regions would cost too much memory and computation time. Furthermore if a single Neural Network (or any other single identification system) is used to identify the system behaviour within all S_i 's the modelling performance severely decreases.

(b) If too large changes in control parameters are allowed, the relationship between parameter changes and their effect on the dynamics become too nonlinear and too difficult to model. Furthermore, that would also mean the loss of the attractive property of small control power of the OGY control. Hence, the choice of K , the number of activation regions, and the magnitude of allowable parameter changes have to be accomplished heuristically by careful observation of the system data and consideration of the performance criteria. The activation regions are symbolically illustrated in Figure 1.

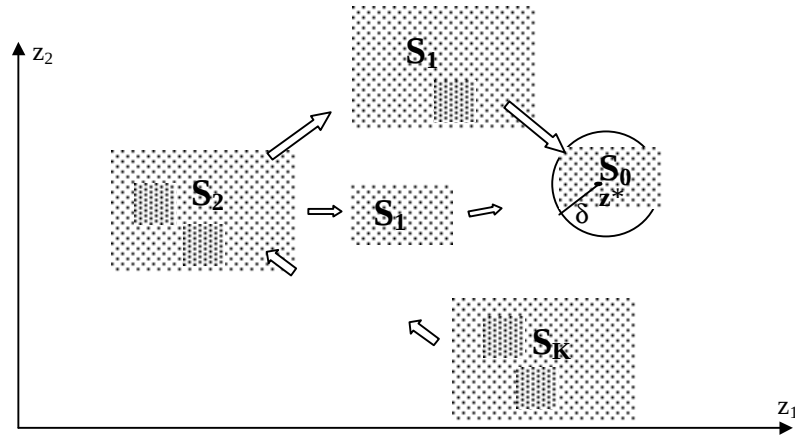


FIGURE 1. Activation regions in the ECR method.

It should be noted that both OGY and ECR methods are only applicable if the chaotic system under consideration starts from initial conditions that lie within the basin of attraction of the strange attractor, which includes the target.

4. THE NEURAL NETWORK BASED REALISATION OF THE ECR

In this work, for the realisation of the ECR approach a Neural Network has been utilised in modelling and control tasks because of its high modelling and intrinsic approximation capability. The specific type of Neural Networks employed will be explained in the following section.

4.1. Radial Basis Functions

The reason behind the preference of a Neural Network using Radial Basis Functions is its high approximation capability. Radial Basis Functions can be viewed as a smooth transition between Fuzzy Logic and Neural Networks, which are very useful tools in Soft Computing.

In the supervised learning problem, the relation between the input and the output of the system is learned from the examples supplied by a *supervisor*. The set of the examples is referred to as the *training set*, and it contains pairs of dependent and independent variables. Let y be the dependent variable and $\mathbf{x}$ is the independent variable with the relation,

$$y = f(\mathbf{x}). \quad (13)$$

In (13) y is a scalar and $\mathbf{x}$ is a vector. Assume that there is a training set $\{[\mathbf{x}_i, y_i]\}$, $i=1, \dots, P$ to be used to approximate the function f . The Radial Basis Function based Neural Network uses a linear model,

$$F(\mathbf{x}) = \sum_{j=1}^m w_j h_j(\mathbf{x}), \quad (14)$$

where w_j is the weight from the j^{th} function to the output and, h is a Radial Basis Function. Radial Basis Functions based Network is a linear combination of the m models of F as illustrated in Figure 2.

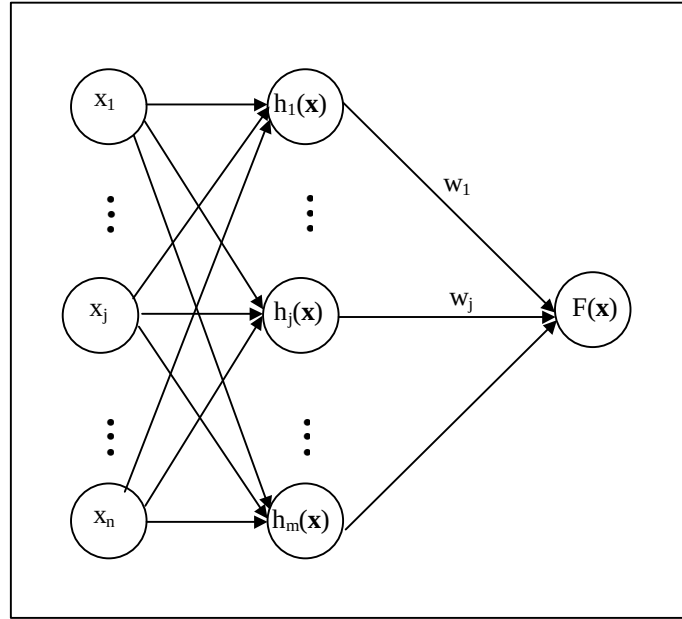


FIGURE 2. Radial Basis Functions Network.

In this network, for a scalar input, the Radial Function can be either a Gaussian Radial Function as given in (15),

$$h(x) = e^{-\frac{(x-c)^2}{r^2}} \quad (15)$$

or a Multi-quadric Radial Function as given in (16),

$$h(x) = \frac{\sqrt{r^2 + (x-c)^2}}{r}, \quad (16)$$

where c is the centre and r is the radius of the function. Since the parameters r and c of the basis functions are assumed to be fixed, the Radial Basis Functions Network is linear in the proposed structure. A Neural Network with a single hidden layer is employed in this work.

Assume that there is a P -pattern training set $\{[x_i, y_i]\}^P$ at hand. The problem is to minimise the sum-squared-error,

$$S = \sum_{i=1}^P (y_i - f(x_i))^2 \quad (17)$$

with respect to the weights of the model. The minimisation of the cost function leads to a set of m simultaneous linear equations with m unknown weights. These equations can be written more conveniently as,

$$\underline{\underline{\mathbf{A}}} \mathbf{w} = \underline{\underline{\mathbf{H}}}^T \mathbf{y} \quad (18)$$

where $\underline{\underline{\mathbf{H}}}$ is the *design matrix*,

$$\underline{\underline{\mathbf{H}}} = \begin{bmatrix} h_1(x_1) & h_2(x_1) & \dots & h_m(x_1) \\ h_1(x_2) & h_2(x_2) & \dots & h_m(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ h_1(x_P) & h_2(x_P) & \dots & h_m(x_P) \end{bmatrix} \quad (19)$$

and $\underline{\underline{\mathbf{A}}}^{-1}$ is the *variance matrix*,

$$\underline{\underline{\mathbf{A}}}^{-1} = (\underline{\underline{\mathbf{H}}}^T \underline{\underline{\mathbf{H}}})^{-1} \quad (20)$$

The solution of the so-called *normal equation* is,

$$\mathbf{w} = \mathbf{A}^{-1} \mathbf{H}^T \mathbf{y} \quad (21)$$

which minimises the sum-squared error. Here, the problem is to choose the number of the radial functions in the hidden layer. To obtain the optimal number of Radial Basis Functions in the hidden layer, the so-called *forward selection* procedure is used. In this procedure, the parameters are the centres and the radii of the Radial Basis Functions. The weight vector is computed after the optimal set of these parameters has been selected. The aim of this procedure is to find a subset of the design matrix $\mathbf{H}$ that consists of fixed parameters. The procedure begins with an empty set. At every iteration a Basis Function, which decreases the predicted error, is added to the subset. During the optimal subset selection, a heuristic *error prediction* is utilised to predict the sum-squared error when the selected Basis Function is added to the subset. The error prediction is held by the so-called *projection matrix*,

$$\mathbf{P} = \mathbf{I} - \mathbf{H} \mathbf{A}^{-1} \mathbf{H}^T, \quad (22)$$

where $\mathbf{I}$ is a $P \times P$ identity matrix. The matrix $\mathbf{P}$ projects the vectors in the P -dimensional space to the m -dimensional subspace. Since there are P training patterns, the training set output $\mathbf{y}$ is in the P -dimensional space. However, the network consisting of m models is in the m -dimensional subspace. According to the least squares principle, the optimal network is the one which has a minimum distance from the projection of $\mathbf{y}$ onto the m -dimensional subspace. By using the projection matrix, the sum-squared error can be calculated as,

$$S = \mathbf{y}^T \mathbf{P}^2 \mathbf{y}. \quad (23)$$

The relationship between $\mathbf{P}_m$, the projection matrix for the m units in the hidden layer, and $\mathbf{P}_{m+1}$, the projection matrix if $\mathbf{f}_j$, the j^{th} column of the design matrix $\mathbf{H}$ is added, is given in (24).

$$\mathbf{P}_{m+1} = \mathbf{P}_m - \frac{\mathbf{P}_m \mathbf{f}_j \mathbf{f}_j^T \mathbf{P}_m}{\mathbf{f}_j^T \mathbf{P}_m \mathbf{f}_j} \quad (24)$$

Forward subset selection utilises this relationship when selecting the basis function that decreases the predicted sum-squared error. This procedure is terminated when a predetermined error criterion is satisfied. More details and the implementation of this procedure in MatLab codes can be obtained from Orr's web page [26].

4.2. Data Gathering

In order to realise the ECR method the dynamics of the system within S_i regions and their dependence on parameter changes have to be sufficiently approximated. Since a Neural Network is employed for this approximation task, the gathered information must be conveniently used in the Neural Network training and control phases. Therefore, a suitable data gathering strategy is followed to ensure that the Neural Network can be trained appropriately, and immediately used for control without any additional analytical computation. In the data gathering phase, the system is evaluated until a sufficient number of data pairs are obtained by changing the parameters within their allowable range Π . The utilised strategy stores the data sets $\{\mathbf{z}_n, \mathbf{z}_{n+1}, \mathbf{p}_n\}$ obtained from the system. Accordingly, the training issue becomes a typical supervised learning. When the data gathering is completed, the parameter map shown in (25) is obtained.

$$\mathbf{p}_n = \mathbf{H}(\mathbf{z}_n, \mathbf{z}_{n+1}) \quad (25)$$

4.3. Training of the Neural Network

The Neural Network has to be trained such that it can perform both targeting and local control tasks. It should be noted that, for any $\mathbf{z}_n$ from the training set it is known which S_i it belongs to. On the other hand, during the application of the ECR method it is not possible to know which activation region the current system state $\mathbf{z}_n$ is in (if at all in any of them), since the Neural Network has an internal representation of the activation regions not available externally. Keeping this very important point in mind, the inputs of the Neural Network and the training scheme has been chosen as follows:

The Neural Network has two input ports, each of dimension N , and an output port of dimension r .

It should be remembered that the training data has been stored as sets of the form $\{\mathbf{z}_n, \mathbf{z}_{n+1}, \mathbf{p}_n\}$. All $\mathbf{z}_n \in S_0$ from the training set are fed to the first port while their next states $\mathbf{z}_{n+1}$ are fed to the second port and the Neural Network is trained to give $\mathbf{p}_n$ at the output port. This part trains the Neural Network for local control, i.e. the network is

trained to give the appropriate control parameter such that $\mathbf{z}_n \in \mathbf{S}_0$ fed to the first port can be driven to the state fed to the second port, which is going to be $\mathbf{z}^*$ during the control mode. Then the remaining $\mathbf{z}_n$'s $\notin \mathbf{S}_0$ are fed to the first port while $\mathbf{z}^*$ is fed to the second port and the Neural Network is trained to give $\mathbf{p}_n$ at the output port. This part trains the Neural Network to supply the appropriate control parameter that will drive any $\mathbf{z}_n \in \mathbf{S}_i$ fed to the first port to $\mathbf{z}^*$ within i steps, ie. to $\mathbf{z}_{n+1} \in \mathbf{S}_{i-1}$ at the next step, hence the network is trained for targeting. Figure 3 illustrates the training scheme.

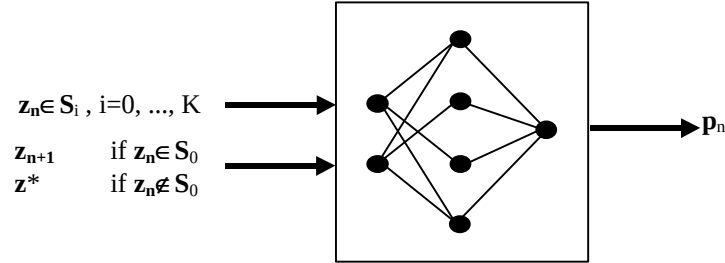


FIGURE 3. Training of the Neural Network for local control and targeting.

4.4. Usage of the Neural Network for Targeting and Control

In the control mode the current system state $\mathbf{z}_n$ is fed to the first port and $\mathbf{z}^*$ to the second port. Depending on $\mathbf{z}_n$, the Neural Network will produce a $\mathbf{p}_{NN}$ that will take the system

- (a) to $\mathbf{z}^*$ if $\mathbf{z}_n \in \mathbf{S}_0$
- (b) to $\mathbf{S}_{i-1}$ if $\mathbf{z}_n \in \mathbf{S}_i$
- (c) to some undefined state if $\mathbf{z}_n \notin \mathbf{S}_i, \forall i=0, \dots, K$

Next it is checked whether $\mathbf{p}_{NN}$ lies within the allowable range and $\mathbf{p}$ is determined according to (26).

$$\mathbf{p} = \begin{cases} \mathbf{p}_{NN} & \text{if } \mathbf{p}_{NN} \in \Pi \\ \mathbf{p}_{nom} & \text{if } \mathbf{p}_{NN} \notin \Pi \end{cases} \quad (26)$$

It should be noted that usually for $\mathbf{z}_n \notin \mathbf{S}_i, \forall i=0, \dots, K$ the Neural Network produces a $\mathbf{p}_{NN}$ that is outside Π such that $\mathbf{p}$ is set to $\mathbf{p}_{nom}$, hence the controller is not activated for $\mathbf{z}_n \notin \mathbf{S}_i, \forall i=0, \dots, K$. The control mode of the Neural Network is depicted in Figure 4.

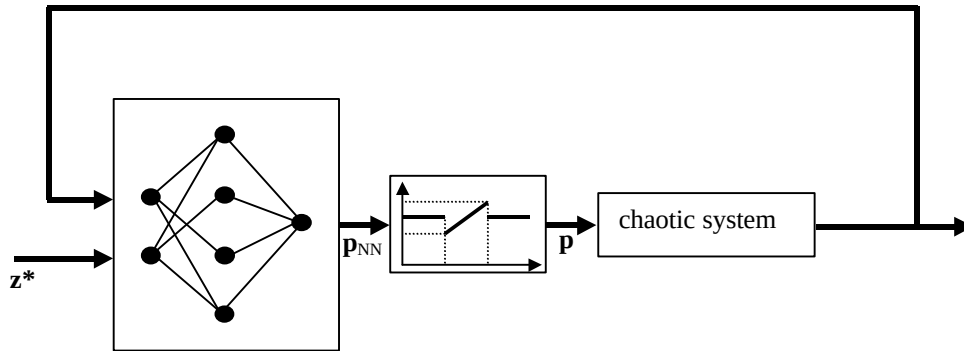


FIGURE 4. Control loop for local control and targeting.

5. RESULTS

The proposed controller has been applied to various chaotic systems and it has been observed that compared to the OGY method, the average reaching time is reduced to 1/3 for the Logistic map, to 1/6 for the Hénon map and to 1/2 for the Lorenz system. In the simulations both noiseless and noisy cases have been considered, where in the noisy case white noise proportional to the noise rate has been added to the measured state(s).

5.1 The Logistic map

The Logistic map is a well known one-dimensional discrete-time system represented by the map,

$$x_{n+1} = px_n(1-x_n), \text{ for } 0 \leq x_n \leq 1, 0 \leq p \leq 4 \quad (27)$$

where p is the parameter of the map. For some ranges of p the map becomes chaotic. For example for $p = p_{nom} = 3.9$, the map exhibits chaotic behaviour, and has an unstable equilibrium point x^* at $x^* = 2.9/3.9$. The controller is activated in S_i regions ($i = 0, \dots, 3$) as can be seen in Figure 5.

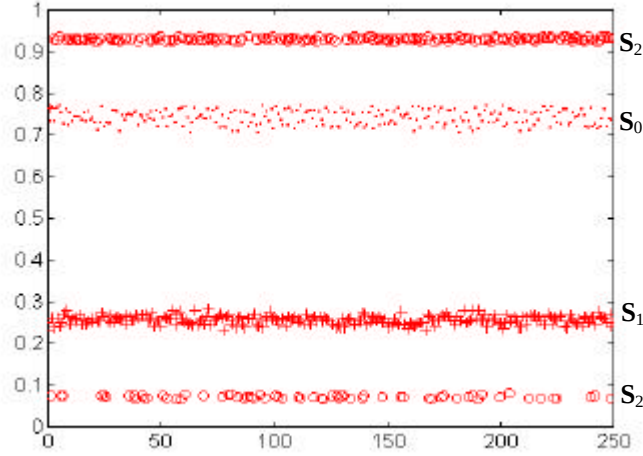


FIGURE 5. Activation regions of the Neural Network Based ECR controller for the Logistic map.

In order to test the average reaching time of the Logistic map controlled by the ECR method, the map has been started from 500 different initial conditions and the number of iterations has been recorded when the system has visited a close neighbourhood of the unstable equilibrium point. In the absence of noise, the average reaching time of the Logistic map is 48.25 iterations when controlled by the Neural Network based ECR controller, whereas it is 143.7 iterations without targeting. In the noisy case, the NN based ECR controller has performed an average reaching time of 54.64 iterations, while OGY without targeting has an average reaching time of 156.8 iterations.

Figure 6 shows the controlled output and the control parameter of the Logistic map in the absence of noise and with a noise rate of 0.264, respectively.

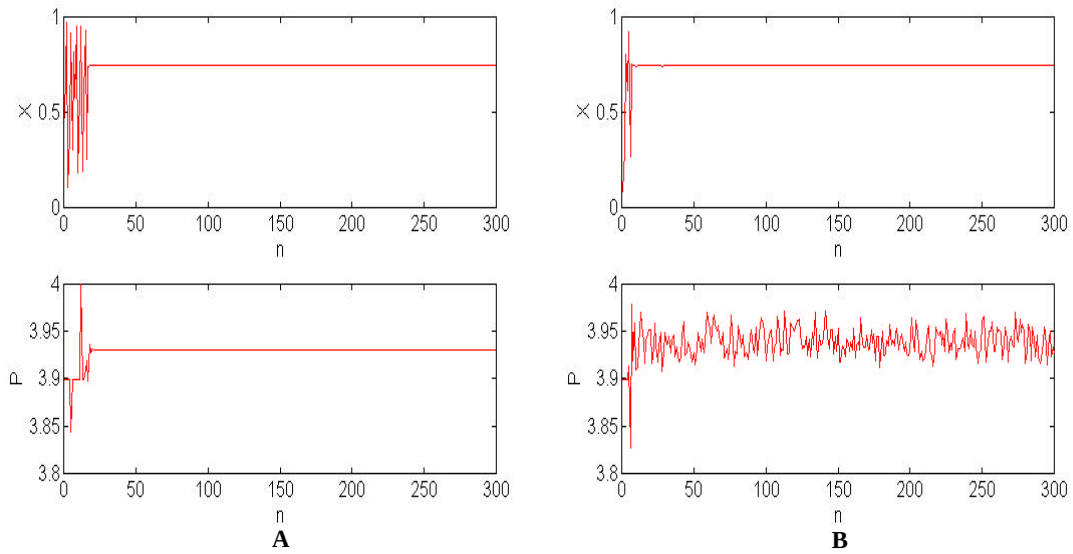


FIGURE 6. (A) The controlled state and the control parameter of the Logistic map in the absence of noise. **(B)** The controlled state and the control parameter of the Logistic map with a noise rate of 0.264.

5.2 The Hénon map

The Hénon map is a two-dimensional discrete-time non-linear system, which is represented by the relations,

$$x_{n+1} = p + 0.3y_n - x_n^2, \text{ for } -2 \leq x_n \leq 2 \quad (28a)$$

$$y_{n+1} = x_n, \text{ for } -2 \leq y_n \leq 2 \quad (28b)$$

where p is the parameter of the map. For some range of p the map behaves in a chaotic manner. For example for $p = p_{\text{nom}} = 1.37$, it becomes chaotic, has a strange attractor, and two unstable equilibrium points at

$$\begin{bmatrix} x_1^* \\ y_1^* \end{bmatrix} = \begin{bmatrix} (\sqrt{0.49 + 4p} - 0.7)/2 \\ (\sqrt{0.49 + 4p} - 0.7)/2 \end{bmatrix} \text{ and } \begin{bmatrix} x_2^* \\ y_2^* \end{bmatrix} = \begin{bmatrix} (-\sqrt{0.49 + 4p} - 0.7)/2 \\ (-\sqrt{0.49 + 4p} - 0.7)/2 \end{bmatrix} \quad (29)$$

The attractor of the Hénon map and the activation regions of the controller are shown in Figure 7. In order to test the average reaching time of the Hénon map controlled by the ECR method, the system has been started from 500 different initial conditions, which lie within the basin of attraction of the strange attractor. The number of iterations has been recorded when the system trajectory visited the close neighbourhood of the unstable equilibrium point. In the absence of noise, the average reaching time of the Hénon map is 103.9 iterations when controlled by the Neural Network based ECR controller, whereas it is 767.9 iterations when no targeting is used. In the noisy case, the NN based ECR controller performed an average reaching time of 134.9 iterations, while the average reaching time is 837.8 iterations in the absence of targeting. Figure 8 shows the controlled output and the control parameter of the Hénon map in the absence of noise and with a noise rate of 0.132, respectively.

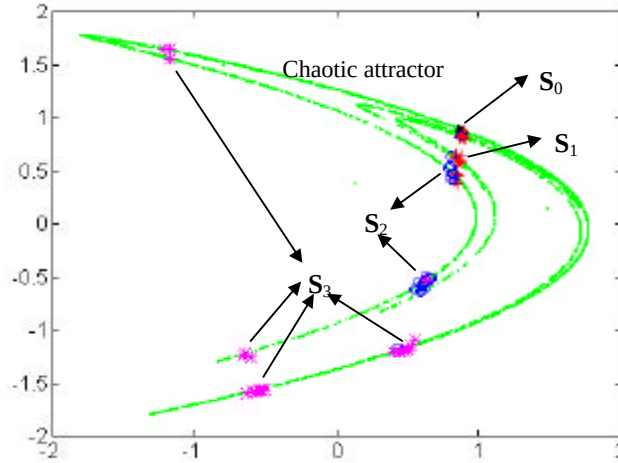


FIGURE 7. Activation regions of the NN Based ECR controller for the Henon map.

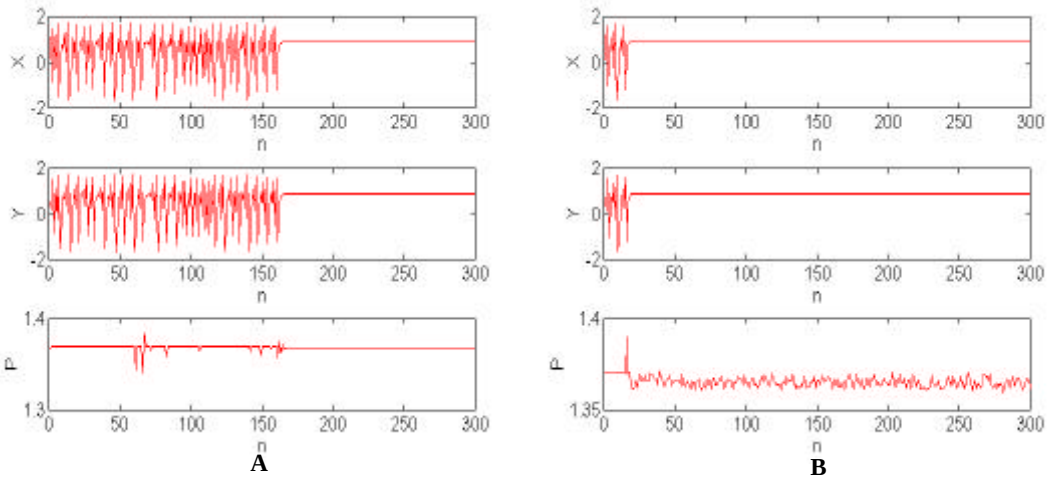


FIGURE 8. (A) Controlled states and the control parameter of the Hénon map in the absence of noise. **(B)** Controlled states and the control parameter of the Hénon map with a noise rate of 0.132.

5.3. The Lorenz system

The Lorenz system is a representative three-dimensional continuous-time chaotic system governed by the equations,

$$\dot{x} = \sigma(y-x), \text{ for } -20 \leq x \leq 20 \quad (30a)$$

$$\dot{y} = (\rho x - y - xz), \text{ for } -50 \leq y \leq 50 \quad (30b)$$

$$\dot{z} = (xy - \beta z), \text{ for } -50 \leq z \leq 50 \quad (30c)$$

where σ , ρ , and β are the system parameters. Again for some parameter ranges the system behaves in a chaotic manner. The nominal values $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$ constitute the most investigated parameter set that corresponds to such chaotic dynamics. In our simulation, only the state y is assumed available for measurement, the Poincaré surface is chosen as $y = 8.48$, and the piercing direction as $dy/dt < 0$. In order to reconstruct the phase space three delayed coordinates are used, where the delay time is taken as $T = 100$ ms. Since delay coordinates are employed, the vector of previous values of the control parameters ($\mathbf{p}_{n-1}$) are also considered and assigned to the input of the Neural Network. Three control parameters are assumed available for control within their allowable range. Figure 9 shows the activation regions of the Neural Network and the attractor of the Poincaré map of the system.

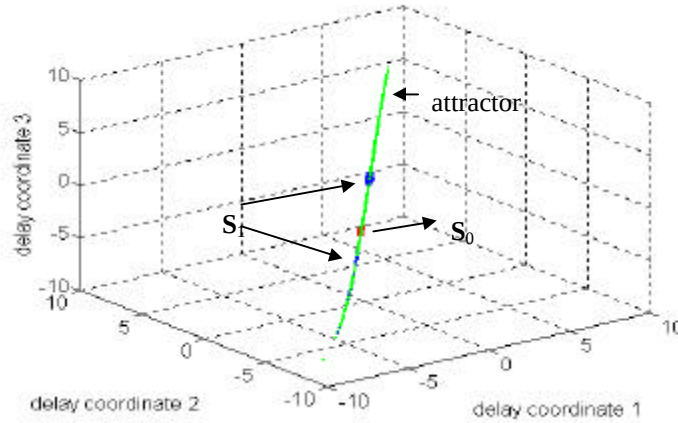


FIGURE 9. Activation regions of the NN Based ECR for the attractor of the Poincaré map of the Lorenz System.

In order to test the average reaching time of the Lorenz system controlled by our method, the system has been started from 100 different initial conditions and the number of piercings has been recorded when it visited a close neighbourhood of the unstable equilibrium point of the Poincaré map. In the absence of noise, the average reaching time of the Lorenz system is 18.31 piercings when controlled by the Neural Network based ECR controller, whereas it is 35.3 piercings without targeting. In the noisy case, the NN based ECR controller performed an average reaching time of 20.08 piercings, while in the absence of targeting the average reaching time was 37.23 piercings. Figure 10 shows the stabilised unstable periodic orbit of the Lorenz system and Figure 11 shows the delay coordinates of the Lorenz system in the absence of noise and with a noise rate of 0.002, respectively.

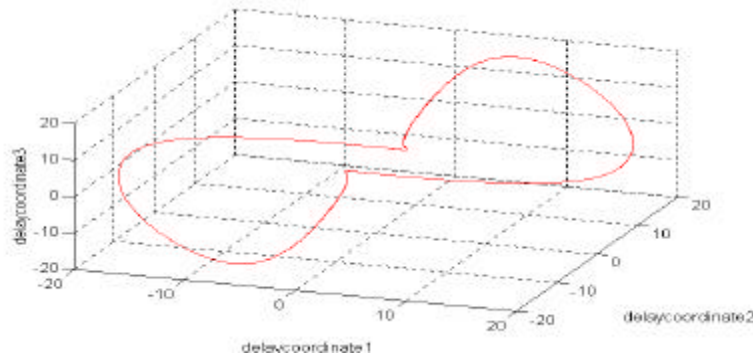


FIGURE 10. The stabilised UPO of the Lorenz system plotted in the delay coordinates space.

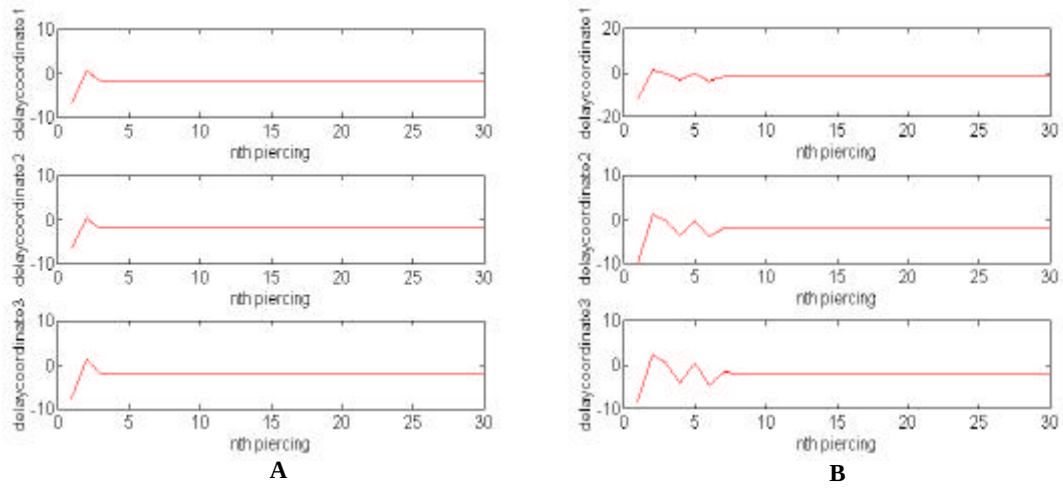


FIGURE 11. (A) Delay coordinates of the Lorenz system in the absence of noise. **(B)** Delay coordinates of the Lorenz system with a noise rate of 0.002.

TABLE 1. Overall Comparison of the Controllers					
			OGY	NN based local control	NN Based ECR
The Logistic map	Noiseless	Training time	X		
		Av. deviation from target		X	
		Av. deviation from p_{nom}		X	
		Average reaching time			X
	Noisy	Training time	X		
		Av. deviation from target		X	
		Av. deviation from p_{nom}		X	
		Average reaching time			X
The Hénon map	Noiseless	Training time	X		
		Av. deviation from target	X		
		Av. deviation from p_{nom}	X		
		Average reaching time			X
	Noisy	Training time	X		
		Av. deviation from target	X		
		Av. deviation from p_{nom}	X		
		Average reaching time			X
The Lorenz system (in real coordinates)	Noiseless	Training time	X		
		Av. deviation from target		X	
		Av. deviation from p_{nom}		X	
		Average reaching time			X
	Noisy	Training time	X		
		Av. deviation from target		X	
		Av. deviation from p_{nom}		X	
		Average reaching time			X
The Lorenz system (in delay coordinates)	Noiseless	Training time	X		
		Av. deviation from target			X
		Av. deviation from p_{nom}			X
		Average reaching time			X
	Noisy	Training time	X		
		Av. deviation from target			X
		Av. deviation from p_{nom}		X	
		Average reaching time			X

Table 1 shows the overall comparison of the classical OGY controller, Neural Network based local (OGY) controller and the Neural Network based ECR controller. The controller that performs best with respect to the corresponding criterion has been marked. The classical OGY method gives the best results with respect to the training time, while the Neural Network based ECR controller gives the best performance with respect to the average reaching time.

6. CONCLUSION

The ECR method presented in this paper, while maintaining the advantages of the OGY method, reduces the average reaching time and is applicable to higher-dimensional systems and multi-parameter control. It should be noted that the same network is used both to model the system dynamics in S_i 's ($i = 1, 2, \dots, K$) and stabilise the system at the target point. Hence whenever K is chosen too large the reaching time to S_0 is appreciably reduced but so is the performance of the controller, i.e. the system after being stabilised at the target may eventually escape from it.

ACKNOWLEDGMENTS

This research has been completed at the Mechatronics Research and Application Centre of Bogazici University.

REFERENCES

1. Ott, E., Grebogi, C., and Yorke, J., Phys. Rev. Lett. 64, No. 11, pp. 1196-1199, (1990).
2. Romeiras, F., Grebogi, C., Ott, E., and Dayawansa, W. P., Physica D 58, pp. 165-192, (1992).
3. Auerbach, D., Grebogi, C., Ott, E., and Yorke, J.A., Phys. Rev. Lett. 69, No. 24, pp. 3479-3482, (1992).
4. Nitsche, G., and Dressler, U., Physica D 58, pp. 153-164, (1992).
5. Rollins, R. W., Parmananda, P., and Sherard, P., Phys. Rev. E 47, No. 2, pp. 780-787, (1993).
6. So, P., and Ott, E., Phys. Rev. E 51, No. 4, pp. 2955-2962, (1995).
7. Ding, M., Yang, W., In, V., Ditto, W. L., Spano, M. L., and Gluckman, B., Phys. Rev. E 53, No. 5, pp. 4334-4344, (1996).
8. Rhode, M. A., Thomas, J., Rollins, R. W., and Markworth, A. J., Phys. Rev. E 54, No. 5, pp. 4880-4887, (1996).
9. Gluckman, B. J., Spano, M. L., Yang, W., Ding, M., In, V., and Ditto, W. L., Phys. Rev. E 55, No. 5, pp. 4935-4942, (1997).
10. Kostelich, E. J., Grebogi, C., Ott, E. and Yorke, J. A., Phys. Rev. E 47, No. 1, pp. 305-310, (1993).
11. Barreto, E., Kostelich, E. J., Grebogi, C., Ott, E. and Yorke, J. A., Phys. Rev. E 51, No. 5, pp. 4169-4172., (1995).
12. Baptista, M., Int'l J. of Bifurcation and Chaos 8, No. 7, pp. 1575-1584, (1998).
13. Elsner, J. B, J. of Phys. A: Math. and Gen. 25, pp. 843-850., (1992).
14. Alsing, P. M., Gavrielides, A., and Kovanis, V., Phys. Rev. E 49, No. 2, pp. 1225-1231, (1994).
15. Solé, R. V., and Prida, L. M., Phys. Lett. A 199, pp. 65-69, (1995).
16. Konishi, K., and Kokame, H., Phys. Lett. A 206, pp. 203-210, (1995).
17. Bakker, R., Schouten, J. C., Takens, F., and Bleek, C. M., Phys. Rev. E 54, No. 4, pp. 3545-3552, (1996).
18. Bakker, R., Korte, R. J., Schouten, J. C., Takens, F., and Bleek, C. M., Fractals 5, No. 3, pp. 523-530, (1997).
19. Konishi, K., and Kokame, H., Physica D 100, pp. 423-438, (1997).
20. Weeks, E., and Burgess, J. M., Phys. Rev. E 56, No. 2, pp. 1531-1540, (1997).
21. Lenz, H. and Obradovic, D., Int'l J. of Bifurcation and Chaos 7, No. 12, pp. 2847-2854, (1997).
22. Yeung, S., "Comparison of Controllers Applied to the Lorenz System", CDS 122 Project, California Inst. of Techn., (1995).
23. Tsonis, A., Chaos; From Theory to Applications, Plenum Press, New York, (1992).
24. Abarbanel, H., Brown, R., Sidorowich, J.J., and Tsimring, L.S., Rev. of Modern Physics 65, No. 4, pp. 1331-1392, (1993).
25. Takens, F., Dynamical Systems and Turbulence, Lecture Notes in Mathematics, Springer Verlag, New York, (1981).
26. Orr, M., <http://www.cns.ed.ac.uk/people/markintro/intro.html>, (1996)